

Atividade 03 — Estado temporal e resiliência

1. Identificação

Estudante: Bárbara Nogueira. Matrícula: 202004744.

Projeto do grupo: Não dê nem água.

Responsabilidade: estado temporal e resiliência, com expiração de leituras, bloqueio da autorização e recuperação.

Situação do relatório: versão preparatória. Execução Wokwi, capturas e link compartilhável ainda pendentes. Os testes locais abaixo não substituem os testes obrigatórios no simulador.

2. Relação com as Atividades 01 e 02

O projeto monitora e irriga plantas domésticas. A Atividade 01 identificou sensores, usuários e dependência de conectividade. A Atividade 02 estabeleceu a necessidade de dados atuais para autorizar atuação. Este protótipo isola a validade temporal da umidade do solo.

O ESP32 mantém estado e executa a regra localmente para permitir sua observação no Wokwi. Nesse recorte, exerce funções de dispositivo e processamento de borda. Não é implementada comunicação de rede. A decisão local não modifica a arquitetura completa registrada na Atividade 02, cuja decisão está na nuvem.

3. Fenômeno, entrada, unidade e faixa

Um potenciômetro representa um índice fictício de umidade de solo entre 0 e 100%. A leitura ADC de 12 bits, entre 0 e 4095, é convertida por valor $\times 100 / 4095$. Não há aquisição, calibração ou validação de umidade física real.

Uma chave representa a interrupção da produção de leituras: esquerda permite amostrar; direita pausa. A pausa não apaga o último valor. A chave não representa a desconexão elétrica real de um sensor. Sua simulação de bounce foi desabilitada para isolar o teste temporal.

Valores não finitos ou fora de [0,100] são inválidos. Digitar i no monitor serial injeta -1 como leitura inválida. Isso testa a lógica de qualidade, sem simular uma falha elétrica específica.

4. Circuito e componentes

ESP32 DevKit v1, um potenciômetro, uma chave deslizante, um LED verde, um LED vermelho e dois resistores de 220 ohms.

Potenciômetro: VCC em 3V3, GND em GND e SIG em GPIO34.

Chave: terminal comum 2 em GPIO23, terminal 1 em GND e terminal 3 desconectado. GPIO23 usa INPUT_PULLUP.

LED verde: GPIO18 → resistor → anodo; catodo em GND.

LED vermelho: GPIO19 → resistor → anodo; catodo em GND.

O LED verde representa autorização lógica de irrigação. Não é uma bomba nem indica água aplicada. O vermelho indica estado desconhecido, obsoleto ou inválido. Em NORMAL e AGUARDANDO, ambos ficam apagados; o estado detalhado aparece no monitor serial.

5. Contrato do evento

O programa produz JSON no monitor serial a 115200 baud. Tipos: `umidade.leitura`, `estado.inicial`, `estado.alterado`, `dispositivo.status` e `leitura.invalida`.

Campos: `eventType` (tipo), `deviceId` (`esp32-barbara-01`), `entityId` (`vaso-01`), `eventTimeMs` (ocorrência em ms desde o início), `sequence` (contador crescente por execução), `value` (último valor válido ou null), `unit` (`pct_simulado`), `state`, `valid`, `actuatorAuthorized` e `ageMs` (idade da última amostra válida ou null). `leitura.invalida` inclui `rejectedValue=-1`.

Exemplo de leitura: `{"eventType":"umidade.leitura","deviceId":"esp32-barbara-01","entityId":"vaso-01","eventTimeMs":3000,"sequence":8,"value":20.00,"unit":"pct_simulado","state":"AGUARDANDO","valid":true,"actuatorAuthorized":false,"ageMs":0}`.

`millis()` não representa UTC. `eventTimeMs` e `sequence` reiniciam em cada execução; não servem para ordenar execuções diferentes sem identidade adicional. Nos eventos de estado e status, `eventTimeMs` é o instante do status/transição, enquanto `ageMs` informa a idade da medição retida. `value` retido não implica dado válido: `valid=false` em estados de falha.

6. Estado, regra e atuação

Estados: `DESCONHECIDO`, `NORMAL`, `AGUARDANDO`, `AUTORIZADO`, `DADO_OBSOLETO` e `LEITURA_INVALIDA`.

Amostragem a cada 1000 ms. Solo fictício seco: valor < 30%. Igual a 30% é `NORMAL`. Três amostras secas válidas consecutivas autorizam a saída; com a amostragem regular, isso requer aproximadamente dois segundos entre a primeira e a terceira. Uma amostra não seca interrompe a contagem.

A leitura vence quando idade >= 5000 ms. O loop verifica a idade continuamente, mesmo sem novas amostras. Ao vencer, preserva o último valor, zera a contagem, entra em `DADO_OBSOLETO`, desliga o verde e liga o vermelho. A ausência inicial mantém `DESCONHECIDO`.

Entrada inválida bloqueia imediatamente a autorização. A recuperação, tanto de inválido quanto de obsoleto, exige novas leituras: uma leitura não seca retorna a `NORMAL`; três secas consecutivas retornam a `AUTORIZADO`. A autorização não é um pulso de bomba; mantém-se enquanto as condições forem válidas. Esse protótipo não implementa controle de volume, cooldown ou histerese da bomba, que pertencem ao recorte de decisão e atuação.

As saídas são escritas de forma idempotente; não há comandos de bomba repetidos. O contador de amostras evita autorização por um pico seco isolado. Os tempos reduzidos facilitam observar o comportamento e não equivalem aos parâmetros de cultivo da Atividade 02.

7. Testes obrigatórios no Wokwi

Para todos os testes, iniciar ou reiniciar a simulação, anotar os tempos efetivamente exibidos e capturar circuito e monitor serial. Os valores aproximados do potenciômetro devem ser confirmados pelo JSON, pois podem variar com a conversão.

Teste normal: iniciar com chave à esquerda e umidade próxima de 60%. Aguardar três amostras. Esperado: `NORMAL`, `valid=true`, autorização false e ambos os LEDs apagados. Registrar estado

inicial, valores, tempos e explicação. Resultado observado no Wokwi: pendente. Evidência: evidencias/teste-normal.png.

Teste decisão: iniciar com 60%, depois ajustar para aproximadamente 20%, mantendo coleta. Esperado: AGUARDANDO nas duas primeiras amostras secas e AUTORIZADO na terceira; verde aceso e vermelho apagado. Retornar para 60% deve retirar a autorização. Resultado observado no Wokwi: pendente. Evidência: evidencias/teste-decisao.png.

Teste adversarial individual (matrícula final 4): obter AUTORIZADO com três amostras de aproximadamente 20%. Registrar o eventTimeMs da última leitura, mover a chave para a direita e manter o potenciômetro em 20%. Observar status sem novos eventos umidade.leitura. Antes de cinco segundos de idade, a informação permanece válida; em idade ≥ 5000 ms, esperado: DADO_OBSOLETO, valor antigo ainda armazenado, valid=false, autorização false, verde apagado e vermelho aceso. Voltar a chave à esquerda deve exigir três novas amostras secas para reautorizar. Resultado observado no Wokwi: pendente. Evidência: evidencias/teste-adversarial.png.

Uma implementação ingênua guardaria 20% indefinidamente e continuaria autorizando irrigação apesar do silêncio. O mecanismo implementado usa o instante da última leitura, expiração independente da amostragem e estado explícito. A saída real deste teste deverá ser registrada após execução no simulador.

Teste adicional de qualidade: digitar i; esperado LEITURA_INVALIDA e bloqueio imediato. Com coleta ativa, uma nova amostra pode chegar em até um segundo e iniciar a recuperação. Pausar antes de injetar facilita observar a falha.

8. Verificação local realizada

A lógica real de controle foi compilada com g++ e testada por asserções. Foram verificados NORMAL, três amostras secas, idade 4999 ms, expiração exatamente em 5000 ms, retenção de valor antigo, recuperação, valores inválidos, NaN, igualdade ao limiar e passagem do contador de tempo por rollover.

O sketch completo também foi compilado e executado em um harness C++ com interfaces Arduino substituídas. Foram verificadas as saídas digitais e analisados os eventos JSON. Logs estão em evidencias/verificacao-local-controle.txt e evidencias/verificacao-local-firmware.txt. Isso comprova a lógica nos cenários locais, mas não comprova compilação para ESP32, conexões do circuito ou execução Wokwi.

9. Limitações

Não são validados umidade física, precisão, calibração, ruído real, instalação, dinâmica de secagem, fluxo de água, segurança elétrica, consumo ou autonomia. Não há nuvem, tomada comercial, Alexa ou aplicativo implementados. O LED indica uma decisão lógica, sem realimentação física. A expiração depende de o processador continuar executando; um travamento real exigiria outras medidas. O estado em RAM reinicia como desconhecido após reboot.

10. Uso de IA generativa

Ferramenta: assistente OpenAI/Codex nesta sessão. Uso: proposta de divisão dos recortes, estrutura do circuito, elaboração do código, documentação e testes locais.

Síntese das orientações: desenvolver um protótipo de plantas domésticas relacionado às atividades anteriores, especializado em expiração de dados e no adversarial de matrícula final 4.

Sugestões adotadas: entrada substituta declarada, chave para interromper amostras, estados explícitos, temporização não bloqueante e eventos estruturados. Adaptação: a saída foi definida como autorização lógica, com regra local restrita ao recorte da simulação.

Limitação identificada: execução local com interfaces Arduino substituídas não valida o Wokwi. Seus logs não foram apresentados como capturas ou resultados observados no simulador. A execução Wokwi e a revisão individual ainda são necessárias. O estudante é responsável por compreender, testar e explicar o código e registrar eventuais alterações posteriores.

11. Link compartilhável e conclusão

Link do projeto Wokwi: pendente de criação e salvamento na conta do estudante.

O relatório deverá ser atualizado com o link, as saídas observadas dos três testes e as evidências antes de montar a versão final do ZIP para o SIGAA.

Referências

Enunciado da Atividade 03 e materiais das aulas 1 a 4.

<https://docs.wokwi.com/guides/esp32>

<https://docs.wokwi.com/diagram-format>

<https://docs.wokwi.com/parts/wokwi-potentiometer>

<https://docs.wokwi.com/parts/wokwi-slide-switch>